

WORKSHOP DE IA

PIPELINE RAG

ARQUITECTURA 4.0

Desglose técnico de funciones: Desde la ingestión de documentos hasta la generación de respuestas precisas.

EL CICLO DE VIDA DEL RAG



INGESTIÓN

Carga de PDF y división en fragmentos semánticos coherentes.



INDEXACIÓN

Vectorización de texto y almacenamiento en base de datos ChromaDB.



RECUPERACIÓN

Búsqueda de similitud para encontrar el contexto más relevante.



GENERACIÓN

Inferencia del LLM utilizando el contexto inyectado en el prompt.

```

1  import os
2  import shutil
3  from dotenv import load_dotenv
4  from langchain_community.document_loaders import PyPDFLoader
5  from langchain_community.vectorstores import Chroma
6  from langchain_core.messages import HumanMessage, SystemMessage
7  from langchain_huggingface import ChatHuggingFace, HuggingFaceEmbeddings, HuggingFaceEndpoint
8  from langchain_text_splitters import RecursiveCharacterTextSplitter
9
10 load_dotenv()
11
12 # =====
13 # CONFIGURACIÓN BÁSICA
14 # =====
15 PDF_PATH = os.getenv("PDF_PATH", "./ana-frank-diario.pdf")
16 CHROMA_DIR = "./chroma_db_clase"
17 EMBEDDING_MODEL = "sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2"
18 HF_MODEL_ID = "Qwen/Qwen3-4B-Instruct-2507"
19
20 SYSTEM_PROMPT = """Sos un asistente experto en análisis documental. Tu único objetivo es responder preguntas basándote EXCLUSIVAMENTE en el contexto proporcionado.
21
22 REGLAS Estrictas:
23 1. IDIOMA: Respondé siempre en español, de forma natural y directa. No incluyas traducciones ni texto en otro idioma.
24 2. CERO ALUCINACIÓN: Si la respuesta no se encuentra en el contexto, no intentes adivinar ni uses conocimiento previo. Respondé EXACTAMENTE: "No encuentro suficiente información en el contexto".
25 3. CERO RAZONAMIENTO: Entregá directamente la respuesta final. Está estrictamente prohibido usar frases introductorias como "Pensando...", "Según el contexto...", "Aquí está la respuesta..." o similares.
26 4. CITAS OBLIGATORIAS: Siempre que el contexto lo permita, incluí la referencia al final de tu respuesta usando el formato [Pág. X, Sección: Y].
27
28 EJEMPLO DE RESPUESTA ESPERADA:
29 La familia de Ana se esconde en el anexo secreto del edificio donde trabajaba su padre, ubicado en Ámsterdam [Pág. 12, Sección: 9 de julio de 1942]."""
30

```

ETAPA 1: INGESTIÓN

PYPDFLOADER

Extrae el texto crudo de los documentos PDF.
Es la puerta de entrada de nuestro conocimiento externo al pipeline.

RECURSIVE SPLITTING

Divide el texto en chunks de 1500 caracteres con un solapamiento (overlap) de 300 para mantener la continuidad narrativa.

"Los textos narrativos requieren ventanas de contexto más amplias para no fragmentar las ideas."

```
# =====  
# ETAPA 1: INGESTIÓN (Carga y Split)  
# =====  
def cargar_y_dividir_documento(ruta_pdf: str):  
    """  
    Carga el PDF y lo divide en fragmentos más pequeños (chunks).  
    """  
  
    print("1. Cargando y dividiendo el documento...")  
    loader = PyPDFLoader(ruta_pdf)  
    paginas = loader.load()  
  
    # AUMENTADO: Textos narrativos necesitan más contexto por chunk  
    splitter = RecursiveCharacterTextSplitter(chunk_size=1500, chunk_overlap=300)  
    fragmentos = splitter.split_documents(paginas)  
  
    return fragmentos
```

ETAPA 2: INDEXACIÓN

-  **Embeddings:** MiniLM-L12-v2 convierte texto en vectores matemáticos.
-  **Persistencia Limpia:** El código elimina versiones previas de la DB para evitar duplicados.
-  **ChromaDB:** Nuestra base de datos vectorial local para almacenamiento eficiente.



```
# =====  
# ETAPA 2: INDEXACIÓN (Vector Store)  
# =====  
def crear_indice_vectorial(fragmentos):  
    """  
    Limpia la base anterior (si existe), convierte los fragmentos a vectores  
    y los guarda en ChromaDB.  
    """  
  
    print("2. Generando embeddings y creando índice vectorial...")  
  
    # CORRECCIÓN: Borrar base de datos vieja para evitar duplicación de chunks en cada ejecución  
    if os.path.exists(CHROMA_DIR):  
        print("    -> Limpiando base de datos vectorial anterior...")  
        shutil.rmtree(CHROMA_DIR)  
  
    embeddings = HuggingFaceEmbeddings(model_name=EMBEDDING_MODEL)  
  
    vectorstore = Chroma.from_documents(  
        documents=fragmentos,  
        embedding=embeddings,  
        persist_directory=CHROMA_DIR  
    )  
    return vectorstore
```

CONFIGURACIÓN CRÍTICA

1500

Chunk Size

0

EQUILIBRIO DEL CONTEXTO

Un tamaño de 1500 caracteres garantiza que el LLM reciba suficiente información para entender el contexto sin saturar la ventana de tokens.

```
RecursiveCharacterTextSplitter(chunk_size=1500, chunk_overlap=300)
```

ETAPA 3: RECUPERACIÓN

- ⚡ **Similarity Search:** Comparamos el vector de la pregunta con los vectores de la base de datos.
- 📚 **Top K (k=5):** Recuperamos exactamente los 5 fragmentos más cercanos semánticamente.
- 🛡️ **Filtro de Relevancia:** Solo lo que el Retriever encuentra llega al LLM.

Función: recuperar_contexto(pregunta, vectorstore, top_k=5)

```
# =====  
# ETAPA 3: RECUPERACIÓN (Retrieval)  
# =====  
def recuperar_contexto(pregunta: str, vectorstore: Chroma, top_k: int = 5):  
    """  
    Busca en la base de datos vectorial los fragmentos más similares a la pregunta.  
    """  
    print(f"3. Buscando información relevante para: '{pregunta}'")  
  
    documentos_recuperados = vectorstore.similarity_search(pregunta, k=top_k)  
    return documentos_recuperados
```

REGLAS DEL SISTEMA (PROMPT)

Regla	Objetivo
Cero Alucinación	Prohibido adivinar. Si no está en el contexto, se responde: "No encuentro información".
Idioma Español	Respuesta natural y directa sin traducciones ni términos en inglés.
Citas Obligatorias	Referenciar siempre [Pág. X, Sección: Y] para dar validez documental.
Sin Razonamiento	El LLM no debe explicar su lógica interna ni decir "Pensando...".

```
# =====
```

```
# CONFIGURACIÓN BÁSICA
```

```
# =====
```

```
PDF_PATH = os.getenv("PDF_PATH", "./ana-frank-diario.pdf")
```

```
CHROMA_DIR = "./chroma db clase"
```

```
EMBEDDING_MODEL = "sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2"
```

```
HF_MODEL_ID = "Qwen/Qwen3-4B-Instruct-2507"
```

```
SYSTEM_PROMPT = """Sos un asistente experto en análisis documental. Tu único objetivo es responder preguntas basándote EXCLUSIVAMENTE en el contexto proporcionado.
```

```
REGLAS ESTRICTAS:
```

1. IDIOMA: Respondé siempre en español, de forma natural y directa. No incluyas traducciones ni texto en otro idioma.
2. CERO ALUCINACIÓN: Si la respuesta no se encuentra en el contexto, no intentes adivinar ni uses conocimiento previo. Respondé EXACTAMENTE: "No encuentro suficiente información en el contexto recuperado para responder con certeza."
3. CERO RAZONAMIENTO: Entregá directamente la respuesta final. Está estrictamente prohibido usar frases introductorias como "Pensando...", "Según el contexto...", "Aquí está la respuesta..." o explicar tu proceso lógico.
4. CITAS OBLIGATORIAS: Siempre que el contexto lo permita, incluí la referencia al final de tu respuesta usando el formato [Pág. X, Sección: Y].

```
EJEMPLO DE RESPUESTA ESPERADA:
```

```
La familia de Ana se esconde en el anexo secreto del edificio donde trabajaba su padre, ubicado en Ámsterdam [Pág. 12, Sección: 9 de julio de 1942]."""
```

ETAPA 4: GENERACIÓN



INFERENCIA CONTROLADA

Utilizamos **HuggingFaceEndpoint** con `temperature=0.0` para asegurar respuestas deterministas y precisas.

✓ **Qwen 3 4B:** Modelo eficiente y potente.

📁 **SystemMessage:** Inyecta las reglas de oro.

```

# =====
# ETAPA 4: GENERACIÓN (LLM + Prompt)
# =====
def generar_respuesta(pregunta: str, documentos_recuperados):
    """
    Arma el prompt inyectando el contexto y le pide al LLM la respuesta.
    """
    print("4. Generando respuesta con el LLM...\n")

    modelo_base = HuggingFaceEndpoint(
        repo_id=HF_MODEL_ID,
        max_new_tokens=256,
        temperature=0.0
    )
    llm = ChatHuggingFace(llm=modelo_base)

    contexto_texto = "\n\n".join([doc.page_content for doc in documentos_recuperados])

    mensajes = [
        SystemMessage(content=SYSTEM_PROMPT),
        HumanMessage(content=f"Contexto:\n{contexto_texto}\n\nPregunta:\n{pregunta}")
    ]

    salida = llm.invoke(mensajes)
    return salida.content

```

INSPECCIÓN CRÍTICA

El bloque de **Debug** permite al desarrollador ver exactamente qué fragmentos recuperó el sistema antes de que el LLM los procese.

```
=== DEBUG: CONTEXTO RECUPERADO ===  
--- Chunk 1 ---  
Nos mudaremos al anexo secreto...  
--- Chunk 2 ---  
El edificio de papá tiene un lugar...
```

¡MANOS A LA OBRA!

Es momento de implementar y probar el pipeline RAG.